Field Hours

CSCE 646 Digital Image

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Sixteen samples in a grid

You store an image as a grid of samples, not as paint. A 4 by 4 gray image holds 16 numbers, one per pixel. You index a pixel by column c and row r, each running from 0 to 3. The value at (c, r) is the light recorded in that cell. You keep the 16 values in row order so pixel (c, r) sits at index 4 times r plus c. Display size is a separate scale.

You can draw those same 16 values as a 2 inch square or a 4 inch square. The samples do not change when only the scale changes. You resample when you need a new grid. An 8 by 8 grid would hold 64 samples, and you do not have them yet. Inventing them by stretching the drawing is a display change, not a new measurement of the scene.

Address the sample first. The size on the page is not another sample.

16 samples, row order

Index is 4r plus c

Display scale is separate

Address of one pixel

1. The image has 4 columns and 4 rows. Both indices run from 0 to 3.

2. Pixel (c, r) = (2, 1) has index 4 times 1 plus 2, which is 6.

3. Changing the drawn width from 2 inches to 4 inches does not move the sample at index 6.

Problem 1. Where is pixel (0, 3) stored in the row-order array of a 4 by 4 image?

Answer. At index 12. The index is 4 times the row plus the column, so 4 times 3 plus 0 is 12.

Problem 2. You draw the same 16 samples at twice the width. How many samples do you have?

Answer. 16. Display scale does not create samples. An 8 by 8 grid would need 64 samples, which this image does not have.

Lecture 2 · 90 minutes

Levels from 0 to 255

You display each sample by mapping it onto a device range. A sample stored from 0 to 255 uses 8 bits, so there are 256 levels. Level 0 is the darkest mark and level 255 is the brightest. You store the nearest level. If the ideal value is 100.4 you store 100, and the leftover 0.4 is quantization error. That error is part of the representation, not a later filter.

You see the error as bands when a smooth ramp has too few levels. A ramp across 4 columns that should rise by a fraction per column cannot, because stored levels are integers. You dither by adding a small pattern before the round, so neighboring pixels round in different directions. Across those 4 pixels the average can sit between two levels even though each stored sample is an integer.

Quantization picks a level. Dither moves the rounding, not the level set.

Ideal sample -> Add a dither -> Round to a level -> Store 0 to 255

Rounding one sample

1. The ideal value is 100.4. The nearest of the 256 levels is 100.

2. The quantization error is 100.4 minus 100, which is 0.4.

3. A neighbor whose ideal value is 100.6 rounds to 101. The pair of errors points in opposite directions.

Problem 1. How many distinct levels can an 8 bit sample store?

Answer. 256. The stored levels run from 0 through 255.

Problem 2. An ideal value is 20.2 and you store the nearest integer level. What is the quantization error?

Answer. 0.2. You store 20, and 20.2 minus 20 is 0.2.

Lecture 3 · 90 minutes

Three channels and a luminance

You store color as three channels, red, green, and blue, each on the same 0 to 255 scale. One pixel is three numbers, so a 4 by 4 color image holds 48 samples rather than 16. You do not mix the channels in the file. A blur that should treat the picture as one image must apply the same weights to each channel. Different weights slide the edges apart and leave a colored fringe.

You convert to luminance when the edit is about brightness. Use weights 0.2, 0.7, and 0.1 on red, green, and blue. Those three weights add to 1. For red 100, green 40, and blue 10, the luminance is 0.2 times 100 plus 0.7 times 40 plus 0.1 times 10, which is 49. You keep the three channels if a later step still needs the hue.

Three channels keep hue. Luminance collapses them with weights that add to 1.

Three channels

Red, green, and blue

48 samples in 4 by 4

Filter each the same

One luminance

Weights 0.2, 0.7, 0.1

The weights add to 1

Hue is dropped

Luminance of one pixel

1. The pixel stores red 100, green 40, and blue 10.

2. The products are 0.2 times 100 = 20, 0.7 times 40 = 28, and 0.1 times 10 = 1.

3. The luminance is 20 plus 28 plus 1, which is 49. The original three channels stay if you still need hue.

Problem 1. How many channel samples are in a 4 by 4 image with 3 channels?

Answer. 48. Each of the 16 pixels stores 3 numbers, and 16 times 3 is 48.

Problem 2. Red is 0, green is 100, and blue is 0. With weights 0.2, 0.7, and 0.1, what is the luminance?

Answer. 70. Only the green term is nonzero, and 0.7 times 100 is 70.

Lecture 4 · 90 minutes

A 3 by 3 kernel

You filter by replacing each pixel with a weighted sum of its neighbors. A 3 by 3 box has 9 weights, each 1/9, and the 9 weights add to 1 so a flat region does not get brighter. You center the kernel on pixel (c, r), multiply each covered sample by its weight, and sum. You write the sum into a new 4 by 4 buffer. Writing into the input would feed changed values to the next pixel.

You need a border rule, because pixel (0, 0) does not have a full neighborhood inside a 4 by 4 image. You can skip the border, copy the input there, or repeat the edge sample so the kernel still sees 9 values. Repeating the edge is a mechanism, not a measurement: the missing sample is a copy. You use the same rule on all 4 borders so the corners are not a special case you forget.

The kernel reads the input. The result goes to a second buffer.

1. Center a 3 by 3 kernel

2. Multiply the nine samples

3. Write a new buffer

4. Use one border rule

A flat region of 2

1. Every sample under the kernel is 2, and each of the 9 weights is 1/9.

2. The weighted sum is 9 times 1/9 times 2, which is 2. The flat region stays flat because the weights add to 1.

3. You store that 2 in the output buffer, so the next pixel still reads the original samples.

Problem 1. A kernel has 9 weights, each equal to 1. What is wrong with using it as a blur on a flat region of value 2?

Answer. The weights add to 9, not to 1, so the region becomes 18. A box blur needs weights that add to 1, such as 1/9 each.

Problem 2. Why do you write filter output into a second buffer?

Answer. So later pixels still read the original samples. Writing into the input mixes old and new values inside one kernel.

Lecture 5 · 90 minutes

Stretch and sharpen

You stretch contrast when the 16 samples of a 4 by 4 image all lie between 40 and 80. Map 40 to 0 and 80 to 255 with a linear rule. A sample of 60 is halfway from 40 to 80, so it maps halfway from 0 to 255, which is 127.5, and you then quantize. You skip this stretch if a single spike already occupies 0 or 255, because the rest of the image would stay cramped.

You sharpen by putting back a fraction of the detail a blur removed. Subtract the blurred copy from the original, then add part of that difference to the original. If a pixel is 80, its blur is 50, and you add half the difference, the result is 80 plus 0.5 times 30, which is 95. You cap the result at 255 so a strong edge cannot wrap around to a dark value.

Stretch uses the range you measured. Sharpen adds a fraction of a difference.

Find low and high -> Stretch toward 255 -> Add high-pass detail -> Cap the result at 255

One stretch and one sharpen

1. Samples lie from 40 to 80. The sample 60 is halfway, so the linear map sends it to 127.5 before rounding.

2. A different pixel is 80 and its blur is 50. The difference is 30.

3. Adding half of 30 gives 95. If a sum exceeded 255 you would store 255, not a wrapped value.

Problem 1. The darkest sample is 40 and the brightest is 80. What does the linear stretch map 40 to, before quantization?

Answer. 0. You send the low end of the measured range to 0 and the high end, 80, to 255.

Problem 2. A pixel is 10 and its blur is 10. You add half the difference. What is the sharpened value?

Answer. 10. The difference is 0, so you add 0. There was no detail to put back.

Lecture 6 · 90 minutes

Pulling samples through a warp

You warp by an inverse map: each output pixel asks where it came from in the input. For each of the 16 output pixels you compute a source location, which may fall between input samples. You read it with bilinear interpolation from the 4 surrounding pixels. You do not push input pixels forward. Two inputs can land on one output and leave another output with no sample at all.

You try a shift of 1 column to the right. Output pixel (c, r) reads the input at column c minus 1 in the same row. Output column 0 asks for column -1, which is outside the 4 columns, so you write 0. Output columns 1, 2, and 3 read input columns 0, 1, and 2. The last input column is never read. That is the inverse map doing one shift, including the hole it creates.

Pushing pixels forward leaves holes. Pulling gives every output one source.

Push forward

Input hunts an output

Holes where none lands

Two inputs can collide

Inverse map

Each output asks once

Sample 4 neighbors

Outside reads become 0

Shift one row by a column

1. One input row is 10, 20, 30, 40.

2. The inverse shift by 1 column reads source columns -1, 0, 1, and 2.

3. The output row is 0, 10, 20, 30. The input value 40 is not copied, because nothing asks for column 3.

Problem 1. Why do you pull samples with an inverse map instead of pushing each input pixel forward?

Answer. Pushing can send two inputs onto one output and leave another output empty. Pulling gives every one of the 16 outputs a single source location.

Problem 2. An inverse map asks for column -1 on a 4 column image. What do you write?

Answer. 0. Column -1 is outside the image, so there is no sample to interpolate.

Lecture 7 · 90 minutes

Alpha on one pixel

You composite two 4 by 4 images with an alpha weight on the foreground pixel. Alpha 1 copies the foreground. Alpha 0 keeps the background. Alpha 0.25 forms the output as 0.25 times the foreground plus 0.75 times the background. If the rest of your pipeline stores premultiplied color, you multiply by alpha before you store, and you stay with that convention. Mixing the two conventions darkens edges.

You compute one pixel by hand. The foreground is 200 with alpha 0.25, and the background is 40 with alpha 1. The straight composite is 0.25 times 200 plus 0.75 times 40, which is 50 plus 30, which is 80. The background alpha does not change the weights because it is already 1. You use that same alpha on all 3 channels so one channel does not pick up a tint.

One alpha blends one foreground pixel over one background pixel.

1. Read color and alpha

2. Read the background

3. Blend with one weight

4. Write the output

Gray under a quarter weight

1. The foreground color is 200 and its alpha is 0.25. The background color is 40.

2. The foreground contributes 0.25 times 200, which is 50.

3. The background contributes 0.75 times 40, which is 30. The output pixel is 80.

Problem 1. Alpha is 1. The foreground is 15 and the background is 90. What is the output?

Answer. 15. Alpha 1 copies the foreground and does not use the background.

Problem 2. Alpha is 0.25, the foreground is 200, and the background is 40. What is the straight composite?

Answer. 80. You add 0.25 times 200 and 0.75 times 40, which is 50 plus 30.

Lecture 8 · 90 minutes

A pipeline on a small grid

You can now address any of the 16 pixels in a 4 by 4 grid, quantize into 256 levels, turn 3 channels into a luminance, blur with a 3 by 3 kernel whose weights add to 1, stretch a range, warp by an inverse map, and composite with alpha. Each step writes a number you can recompute on this grid. Staying at 4 by 4 is practice, not a limit on later images.

You chain two operations only after you name the buffer each one reads. A blur must finish into its own buffer before a warp samples that result. A composite that expects alpha from 0 to 1 is wrong if you feed it a sample you already stretched toward 255. You write the range on every buffer. The mechanism is the same either way. The range is what changes the meaning.

Name the buffer and its range before you connect the next operation.

Samples and levels -> Filter or stretch -> Warp by inverse map -> Composite with alpha

Blur a flat foreground, then place it

1. A 3 by 3 box on a flat region of 2 returns 2, because the weights add to 1. You write 2 into a new buffer.

2. That buffer is the foreground. Alpha is 0.25 and the background is 40.

3. The composite is 0.25 times 2 plus 0.75 times 40, which is 0.5 plus 30, which is 30.5.

Problem 1. Combine the box filter and the composite. A flat foreground of 2 is blurred with weights that

add to 1, then placed at alpha 0.25 over a background of 40. What is the output?

Answer. 30.5. The blur leaves the value 2, and 0.25 times 2 plus 0.75 times 40 is 0.5 plus 30.

Problem 2. Pixel (2, 1) is warped after a stretch that maps the sample 60 to 127.5. Which value does the inverse map read?

Answer. The stretched value, once that stretch has been written into the buffer the warp reads. The inverse map samples the buffer you give it. It does not remember the earlier range of 40 to 80.